성장하는 신입 개발자 안상문입니다.

Frontend Portfolio | Ahn Sang-moon

201612648 안상문

INDEX

01 자기소개

02 기술스택

03 레퍼런스

04 개발 경험

1. 자기소개

1. 자기소개

- 프론트엔드를 주로 했지만 백엔드까지 섭렵하여 풀스택 개발자가 되는 것이 첫 번째 목표입니다.

- 프로젝트 최적화에 가장 많이 고민하며 힘써 사용자가 항상 최고의 사용자 경험을 가지도록 노력하고 있습니다.

- Cloud Firestore, Sanity 등의 데이터베이스 도구를 사용하여 직접 스키마를 작성하여 Next.js 및 React 를 이용한 서버리스 기반의 풀스택 개발의 경험이 있으며, 위의 경험을 토대 데이터 모델링 및 관리 경험을 쌓았으며, RestAPI 및 프로젝트 리더 경험을 토대로 협업 역량을 갖추었습니다.

- 아직 부족한 점이 많은 신입 개발자이지만 모르는 것을 묻는 것에 주저하지 않고 처음 접하는 스택을 접하는 것에 두려움이 없습니다.

부경대학교 졸업
컴퓨터공학과 [3.75 / 4.5]

Contact

Tel : 010-4165-0073

github : https://github.com/ansm0403
Blog : https://velog.io/@tkdans312/posts
E-mail : kirianir@naver.com / tkdans312@gmail.com

2. 기술 스택

1. 기술스택

언어

Javascript

프로토타입과 클로저 등 기본 개념들을 모두 이해하고 있습니다. 특히 가장 까다로웠던 동기/비동기 개념에 대해서는 블로그에 여러 포스트를 작성할 만큼 열심히 공부했습니다.

Typescript

완벽하다고 할 순 없지만 기본 문법을 충분히 이해하고 있고, 조건부 타입과 제네릭을 이용한 타입추론 등을 사용하여 프로젝트에 무리 없이 적용할 수 있습니다.

데이터 관리

Sanity

Sanity를 통해 댓글과 좋아요 등의 커뮤니티 기능을 구현했으며, 페이지네이션 기능을 추가했습니다.

Cloud firestore

Firebase의 Firestore를 활용하여 실시간 데이터 업데이트 기능과 계좌 애플리케이션의 트랜잭션을 구현했습니다.

prisma + supabase

숙련도가 낮은편이지만 간단한 프로젝트에서 공공데이터를 이용하여 데이터 관리 및 댓글, 좋아요 기능을 구현하였습니다.

프론트엔드

React / Next.js

SPA 의 특징을 이해하고 있으며, Next.js 를 통한 SSR과 CSR의 장단점을 이해하여 상황에 적합한 컴포넌트를 작성했으며, 풀스택 라이브러리인 Next.js의 장점 중 하나인 API route를 자주 활용했습니다.

Emotion-Styled / Tailwind CSS

Emotion을 활용하여 별도의 디자인 시스템을 구축하고 이를 프로젝트에 성공적으로 적용했으며, Tailwind css 의 숙련도는 높지 않지만 인스타그램과 웹툰 프로젝트에 성공적으로 적용하였습니다.

상태관리, 배포 및 협업

Redux

React의 useReducer로 상태 관리의 기초를 익힌 후, 호텔 예약 프로젝트에서 Redux를 적용했습니다. Redux Toolkit을 활용해 상태 흐름을 명확히 관리하여 유지보수성과 협업 효율을 높였습니다.

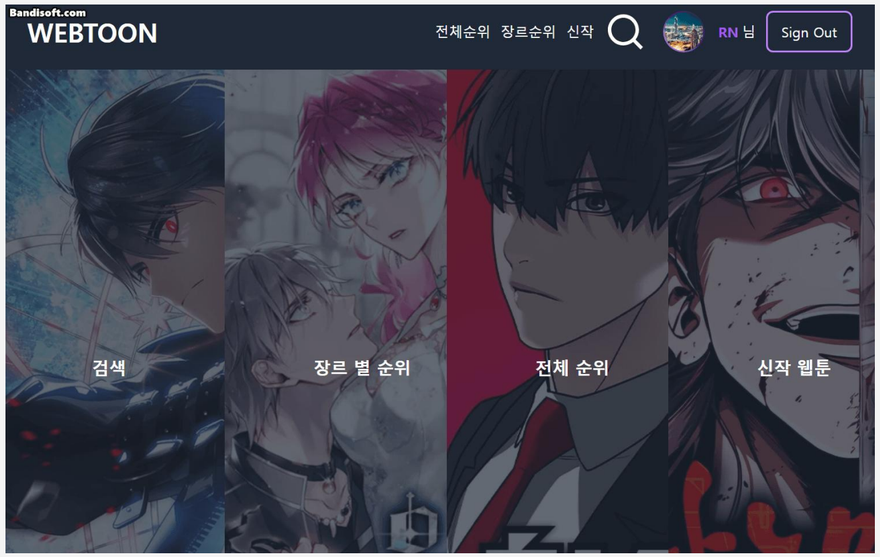
REST API

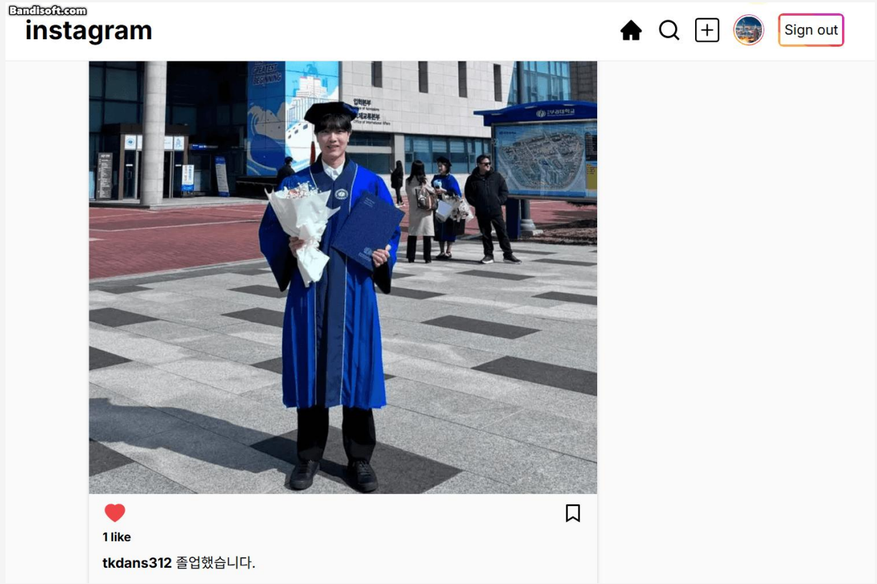
웹툰 프로젝트에서 자바 스프링 백엔드 개발자와 협업하여 Swagger + REST API를 활용한 개발 경험이 있습니다.

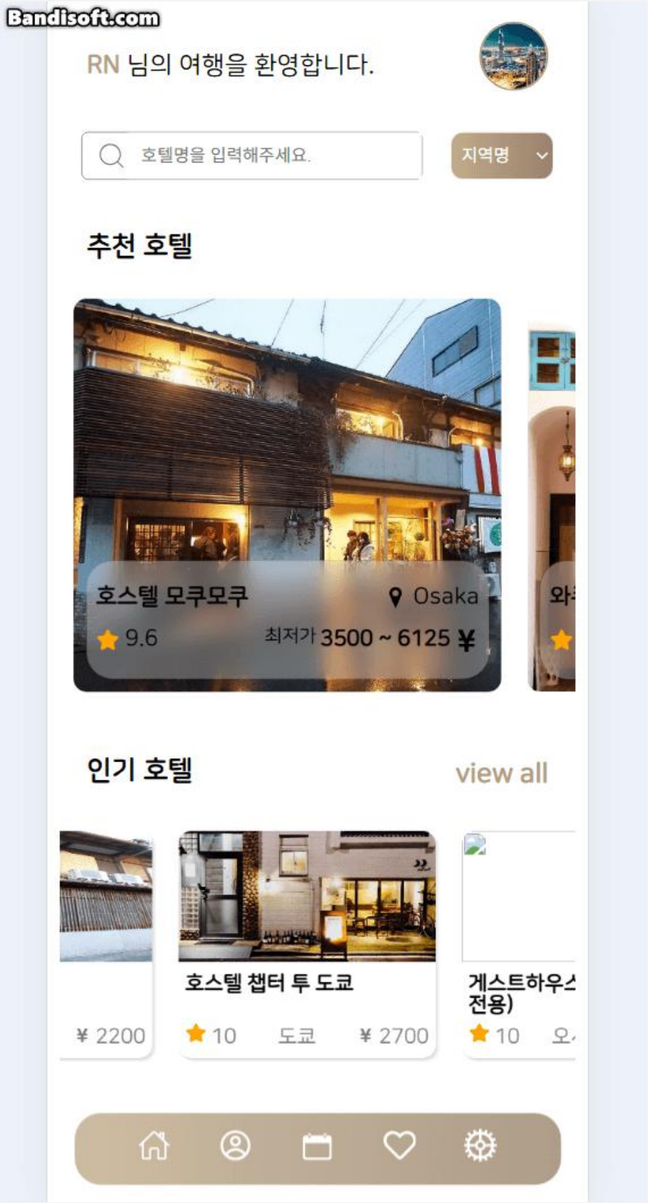
Github Actions CI/CD

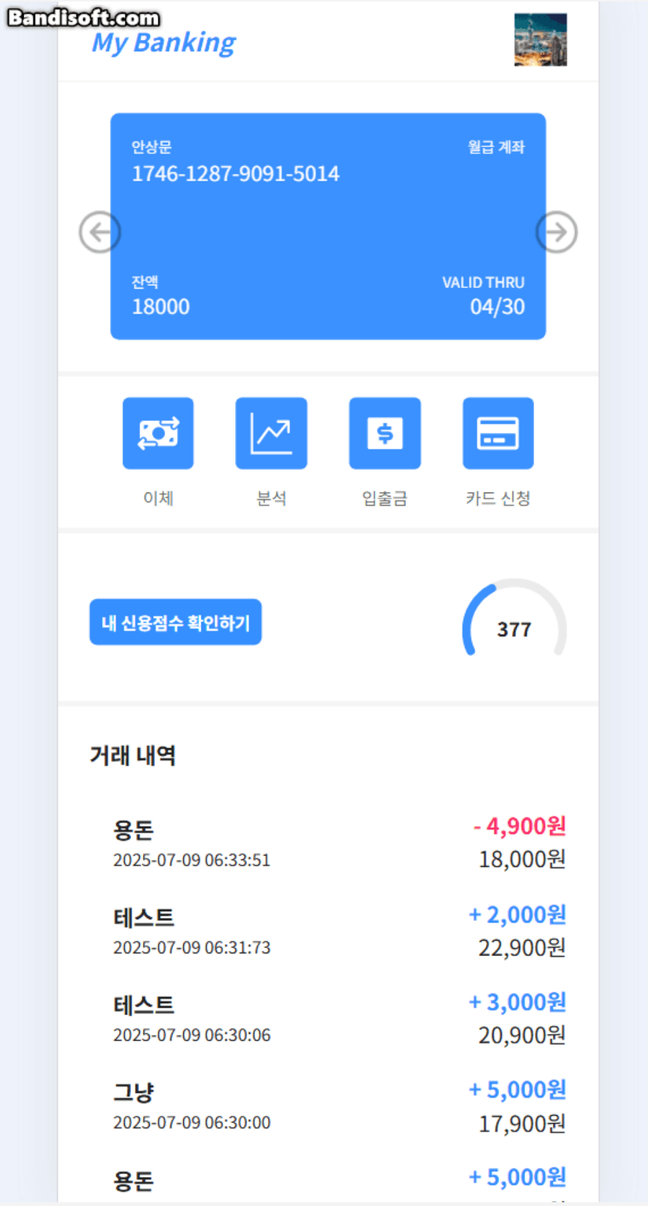
프로젝트들을 vercel 로 배포할 때 Github actions CI/CD 로 자동화 하였습니다.

3. 레퍼런스


Bandisoft.com
WEBTOON
전체순위 장르순위 신작
RN 님
Sign Out
검색
장르 별 순위
전체 순위
신작 웹툰


Bandisoft.com
instagram
Sign out
1 like
tkdans312 졸업했습니다.


Bandisoft.com
RN 님의 여행을 환영합니다.
호텔명을 입력해주세요.
지역명
추천 호텔
호스텔 모쿠모쿠
Osaka
9.6
최저가 3500 ~ 6125 ¥
인기 호텔
view all
¥ 2200
호스텔 챕터 투 도쿄
10
도쿄
¥ 2700
게스트하우스
전용)
10


Bandisoft.com
My Banking
안상문
월급 계좌
1746-1287-9091-5014
잔액
18000
VALID THRU
04/30
이체
분석
입출금
카드 신청
내 신용점수 확인하기
377
거래 내역
용돈
2025-07-09 06:33:51
- 4,900원
18,000원
테스트
2025-07-09 06:31:73
+ 2,000원
22,900원
테스트
2025-07-09 06:30:06
+ 3,000원
20,900원
그냥
2025-07-09 06:30:00
+ 5,000원
17,900원
용돈
+ 5,000원

4. 개발경험

4. 개발경험

#1. 대한민국 모든 웹툰들을 긁어모아 랭킹을 매긴 웹툰 웹 서비스 프로젝트

개발 목적

현재 대한민국에서 가장 인기있는 컨텐츠인 웹툰을 하나의 웹에 모아 조회수에 따른 랭킹을 매겨 여러 플랫폼의 웹툰에 쉽게 접근할 수 있도록 만드는 서비스입니다.

나의 역할 팀 리더, 프로젝트 전체 기획 및 프론트엔드 구현
팀원의 역할 자바, Redis, JWT 를 이용한 백엔드 서버 구축, 파이썬을 이용한 웹툰 정보 크롤링

기술스택

백엔드 : Java / Python / Spring / JWT / Redis / MySQL
프론트엔드 : Next.js / TypeScript / Tailwind CSS / SWR

배포 : https://webtoon-web-service-renew.vercel.app/
Github : https://github.com/ansm0403/WebtoonWebService

프로젝트를 통해 얻은 것

- 프로젝트 시작 당시 저는 임베디드에서 프론트엔드로 넘어온지 얼마 되지 않았고, 팀원 두 명 중 한 명은 백엔드만, 다른 한 명은 모바일앱 개발만 공부한 상태여서 프론트엔드 개발과 저 자신은 웹개발에 대한 지식이 많이 부족했습니다.
- 하지만 제가 전체 아이디어를 구상했기에 팀원들과의 원활한 소통을 위해 백, 프론트 관련 지식을 학습했고, 그 후 유스케이스 다이어그램을 먼저 작성하여 `팀원들에게 아이디어를 명확하게 전달했습니다. 이를 기반으로 클래스 다이어그램과 시퀀스 다이어그램을 완성했습니다.
- Swagger 를 통한 API 문서화로 백엔드 개발자와 효율적으로 소통할 수 있었습니다.
- 해당 프로젝트에서 가장 어려웠던 점은 자바 스프링과의 연동으로 enum 으로 표시한 성별이 자바 내에서 정수로 바뀌어 명세서랑 불일치 했던 점부터 스프링 코드내에서 Cross Origin 설정까지 여러 부분이 굉장히 어려웠습니다.

4. 개발경험

#1. 대한민국 모든 웹툰들을 긁어모아 랭킹을 매긴 웹툰 웹 서비스 프로젝트

최적화

- 웹툰 목록은 페이지네이션으로 구현되어있기에 첫 로드시 페이지네이션 컴포넌트가 먼저 렌더링 되어 좋지 않은 CLS 점수(0.6)를 보임.

  - 각 웹툰들의 스켈레톤 UI 를 만들어서 CLS 최적화
  - 웹툰 디테일 페이지에 추가적인 스켈레톤 UI 로 CLS 최적화. CLS 가 0.08까지 감소

- 웹툰 목록 첫 번째 페이지의 프리페칭

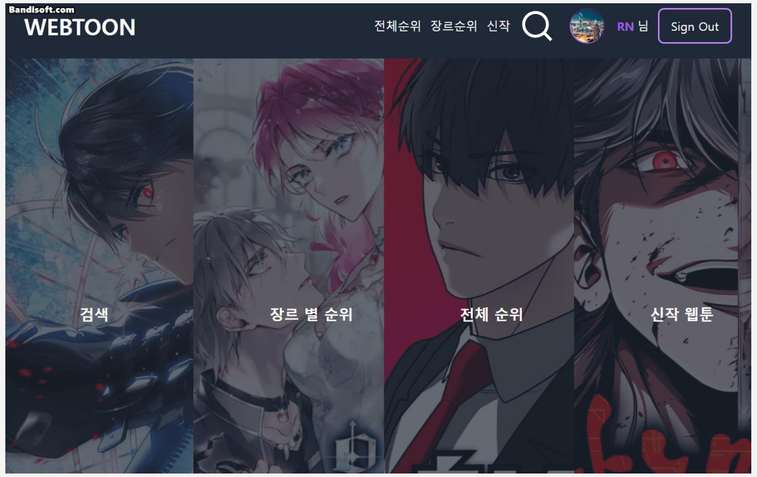
  - 웹툰 목록들에서 2, 3번째 페이지는 몰라도 첫 번째 페이지는 목록에 접근하면 무조건 봐야한다.
  - 따라서 각 목록들의 첫 번째 페이지들을 프리페칭하여 더 빠른 페이지 접근을 가능하게 하였다.
  - DOMContentLoaded : 422ms → 356 ms / 로드 : 506ms → 403 ms

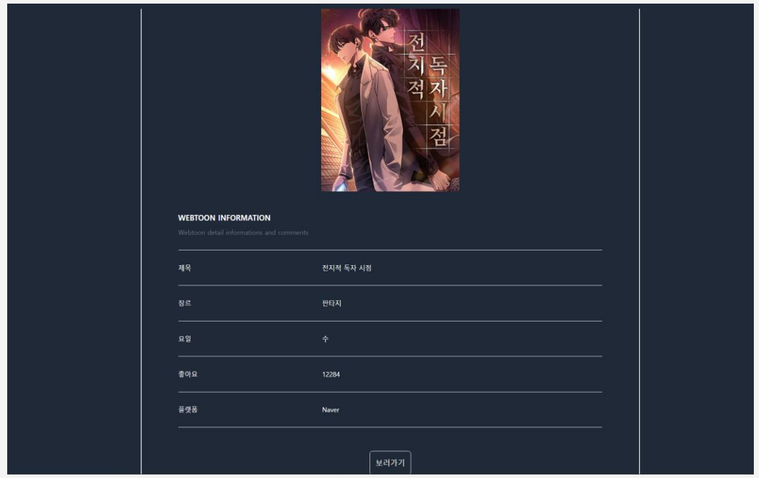
- 해당 프로젝트의 홈은 데이터 요청이 없어 오래 걸리지 않을거라는 예상과 달리 첫 번째 페이지의 LCP 가 2.1 가량으로 생각보다 좋지 않는 성능을 보임.

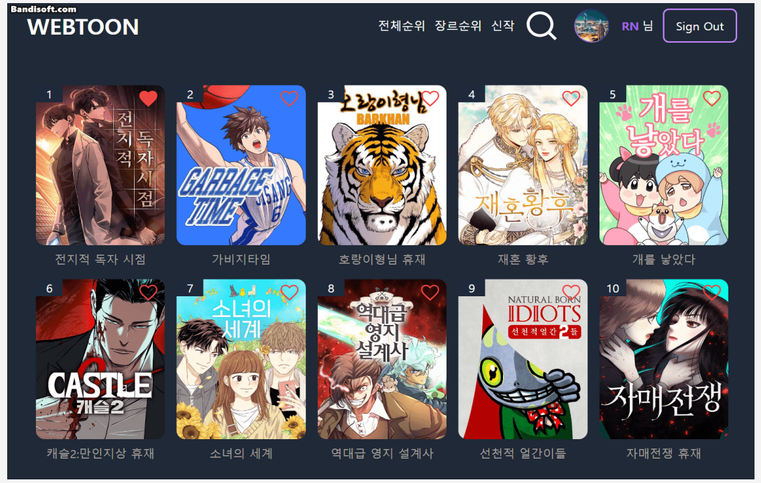
  - 개발자 도구의 성능 탭을 통해 확인한 결과 layout.tsx 의 다운로드 시간이 길었던 것을 확인
  - 코드 스플리팅을 통해 해당 파일의 컴포넌트들을 동적 임포트.
  - 모바일 navbar 는 메뉴 버튼을 누르지 않으면 나오지 않기에 스플리팅
  - 추가로 확인해보니 다른 컴포넌트들보다 react-icons 에서 가져온 search Icon 이 생각보다 매우 큰 크기를 가짐.
  - 해당 파일까지 스플리팅한 후 LCP 는 0.21 로 매우 높은 성능 개선 효과를 보임.

4. 개발경험

#1. 대한민국 모든 웹툰들을 긁어모아 랭킹을 매긴 웹툰 웹 서비스 프로젝트







4. 개발경험

#2. 일본으로 여행을 떠나요! 일본 호텔 예약 어플리케이션

개발 목적

네이버에서 미용실을 예약하는 도중에 이 예약이란 것은 어떤 로직과 상태로 구성되며, 생각보다 구현이 어려울 것 같다는 생각에 만들어보게 되었습니다.

나의 역할

- 프로젝트 기획, 프론트엔드 및 서버리스 백엔드

기술스택

백엔드 : Cloud Firestore
프론트엔드 : React / Javascript / TypeScript / Emotion-styled / React-query
개발 도구 / 협업 / 배포 : GitHub, GitHub Actions (CI/CD), Vercel

배포 : https://hotels-app-sage.vercel.app/
Github : https://github.com/ansm0403/Hotel-reservation

프로젝트를 통해 얻은 것

- 저는 현실에서 예약기능을 사용할때에는 매우 간편하게 사용했지만 그 과정은 굉장히 복잡했습니다.
  - 단순히 모든 요일에 예약이 가능하거나
  - 특정 날짜에 예약이 불가능한 경우
  - 시간표를 시스템에 보관해도 되는 경우
  - 그렇지 않은 경우

등 많이 복잡했습니다. 많은 경우에 대해 텍스트로 공부는 했지만 직접 구현해보진 못했습니다. 하지만 직접 구현하면서도 많은 경험과 상황에 대해 공부할 수 있었습니다. 호텔의 경우는 방을 사용자가 직접 지정하는 것이 아니었기에 단순히 남은 방만 있으면 예약할 수 있어 제약 조건이 생각보다 적었습니다.

4. 개발경험

#2. 일본으로 여행을 떠나요! 일본 호텔 예약 어플리케이션

최적화

- **무거운 캐러셀 컴포넌트 (지연 로딩으로 렌더링 최적화)**

  - 초기화면의 추천 호텔 캐러셀에는 총 20개의 호텔리스트가 들어있습니다.
  - 하지만 모바일 기준 3개의 호텔만 캐러셀에 드러나기 때문에 나머지 17개의 데이터가 낭비되고 있었습니다.
  - 그래서 react-lazy-load-image 패키지를 이용하여 화면에 나타나는 이미지만 우선 렌더링 되도록 만들었습니다.
  - 그 결과 LCP가 2.1초에서 1.81 로 감소하였습니다.
  - 하지만 최적화의 결과로 캐러셀이 자동으로 움직이는 경우에는 화면에 처음 드러날때 잠시 이미지가 비어있는 상태로 사용자 경험에 조금 부정적일 수도 있을거라 생각하여 어떤 것이 좋을지 고민할 필요가 있을 것 같습니다.

- **검색 시 서버에 연속적인 요청 (검색 디바운스)**

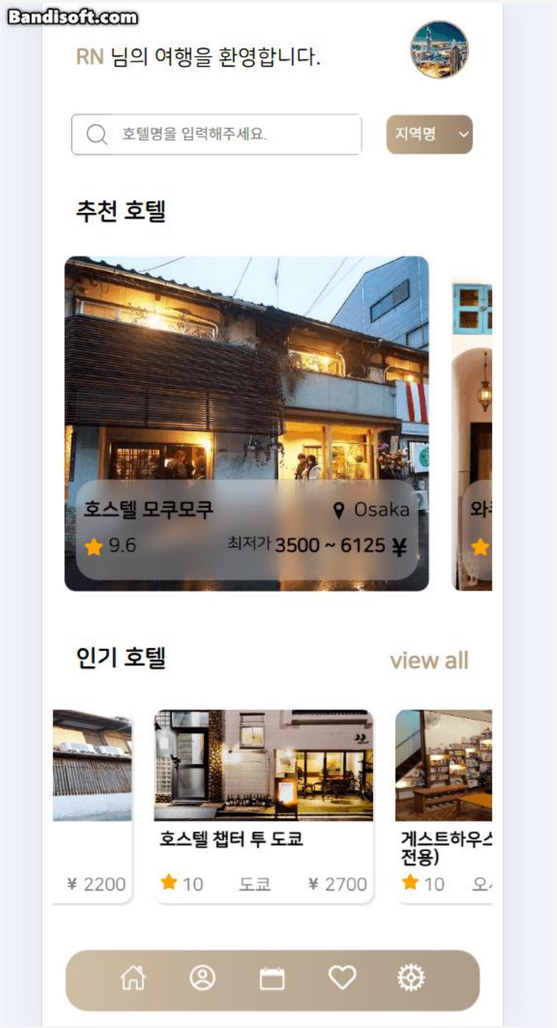
  - 검색 기능에서 키워드를 입력하면 그 키워드에 해당하는 모든 결과가 나열되도록 만들었는데 한 글자가 입력되거나 삭제될 때마다 즉 키워드가 변경될 때마다 네트워크 요청을 하는 문제가 발생하였습니다.
  - 이것을 해결할 수 있는 방법으로 throttle 과 debounce 가 있었고, 이 중 debounce 를 사용 하여 네트워크 요청을 한 번만 하도록 만들었습니다.

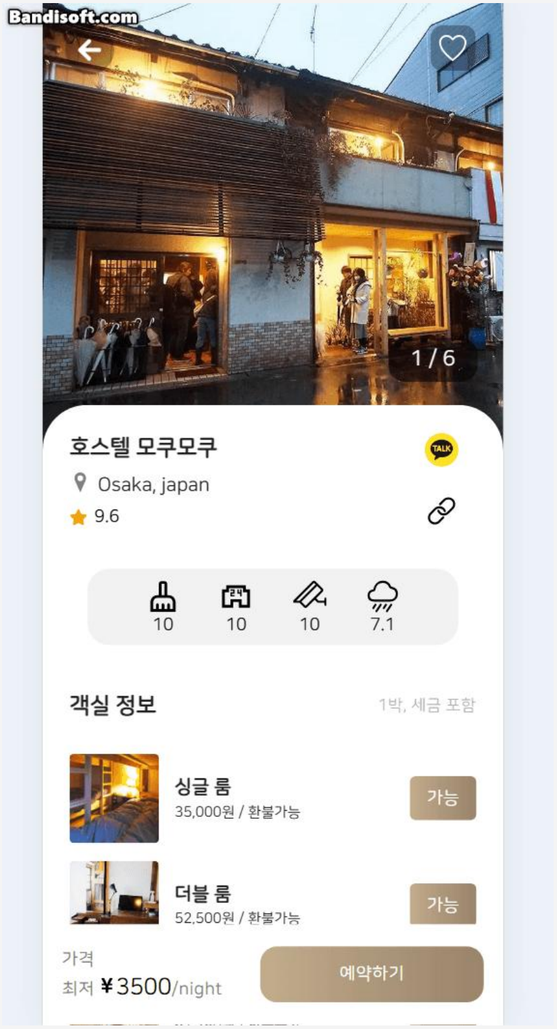
- **호텔 상세페이지 intersection Observer 로 최적화**

  - 호텔 상세페이지에는 많은 정보가 담겨있고 스크롤을 아래까지 내려야합니다.
  - 아래로 내린 곳에는 추천 호텔과 댓글 컴포넌트가 존재하는데, 이 컴포넌트들은 네트워크에 데이터를 요청합니다.
  - 위의 지연로딩과 마찬가지로 뷰포트에 들어올 때 서버에 요청을 하여 최적화를 수행합니다.
    이때 intersection Observer를 통해 뷰포트에 들어올 때 react-query 의 enabled 를 true 로 만들어 데이터를 불러옵니다.
  - LCP 가 1.90 에서 1.74 로 감소하였습니다.
  - 하지만 스크롤을 내리면 데이터를 불러오면서 갑자기 스켈레톤 UI가 나타나는데 저는 단 한 번도 어플리케이션들을 사용하면서 이러한 경험을 해본 적이 없습니다.
  - 차라리 사용자 경험을 위해 이러한 부분을 지우고 최대한 많은 컴포넌트를 스플리팅하여 LCP 를 1.60까지 감소시켰습니다.

4. 개발경험

#2. 일본으로 여행을 떠나요! 일본 호텔 예약 어플리케이션





4. 개발경험

#3. 돈이 오가는 계좌 어플리케이션

개발 목적

동창 중에 금융업에서 일하고 있는 친구가 있습니다. 전에 친구의 추천을 받아 이력서를 넣기 전 금융업에서 사용하는 어플리케이션에 대해 이해가 필요할 것 같아 구현해보게 되었습니다.

나의 역할

- 프로젝트 기획, 프론트엔드 및 서버리스 백엔드

기술스택

백엔드 : Cloud Firestore
프론트엔드 : Next.js / Javascript / TypeScript / Emotion-styled / React-query
개발 도구 / 협업 / 배포 : GitHub, GitHub Actions (CI/CD), Vercel

배포 : https://bank-accounts-zeta.vercel.app/
Github : https://github.com/ansm0403/Account_app

프로젝트를 통해 얻은 것

- 실시간 업데이트를 사용하는 방법과 리액트 쿼리 캐시와 합쳐서 구현하는 방법을 알게 되었습니다.
- 은행 어플리케이션의 주 기능인 입출금/이체는 단순히 데이터를 주고 받는 것이 아니라 트랜잭션 데이터가 추가되며 이 모든 과정을 하나의 트랜잭션으로 만들어야 했기에 해당 과정을 수행하면서 다시 공부가 됐으며 많은 것을 얻을 수 있었습니다.

4. 개발경험

#3. 돈이 오가는 계좌 어플리케이션

최적화

- NoSQL 인 Firestore 데이터베이스에서 관계 데이터 보관 방식을 참조에서 데이터 복제로 변경

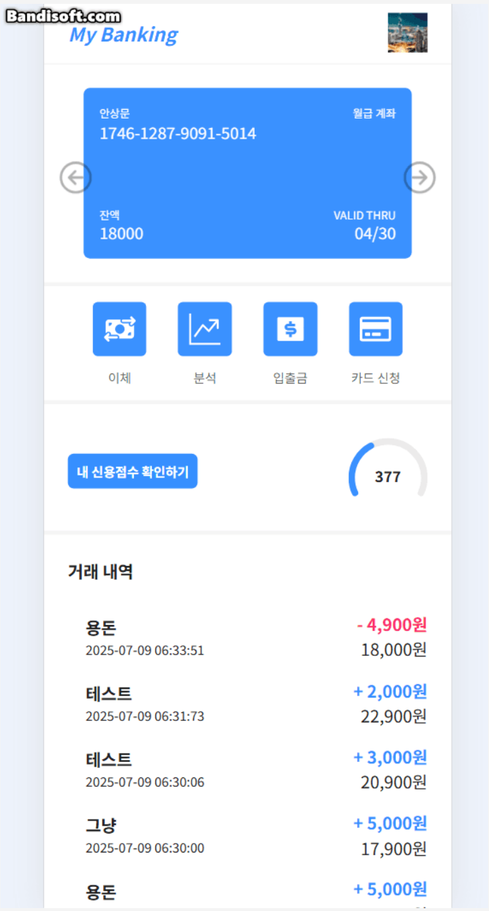
  - 유저 컬렉션에 존재하는 하위 컬렉션인 트랜잭션의 경우 트랜잭션 컬렉션의 ID 를 그대로 받아서 참조로 저장했습니다.
  - 하지만 이렇게 될 경우 하위 컬렉션을 받아온 후, 또 받아온 참조들을 이용하여 서버에 추가 요청하는 방법 말고는 없었습니다.
    (Sanity 에서는 이것을 쿼리 한 줄로도 해결할 수 있었는데 아쉬웠습니다.)
  - NoSQL 은 원래가 데이터 복제를 권장하는 방식으로 만들어졌고, 쿼리의 수를 줄이는 것이 성능에 가장 큰 영향을 준다는 것을 알게 됐습니다.
  - RDB 처럼 만들려 했던 것이 패착으로, NoSQL 본연의 방식대로 데이터 복제로 바꿔 참조가 아니라 데이터 그 자체를 하위 컬렉션에 추가했습니다.
  - 10개의 트랜잭션을 가져오는 기준 280ms → 132ms 로 개선하였습니다.

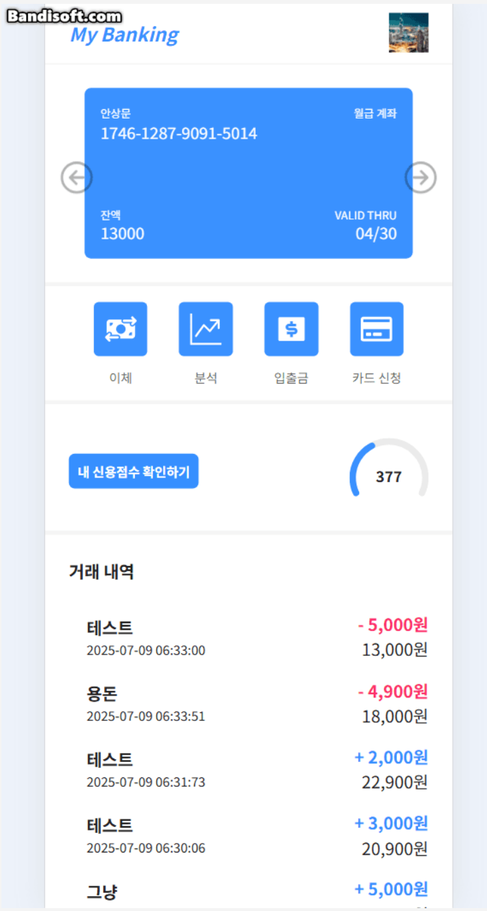
- 메인 페이지의 굉장히 큰 LCP 지표. (서버 컴포넌트에서 클라이언트 컴포넌트로)

  - 해당 홈 의 LCP 가 굉장히 크게 나왔습니다. 사실 어플리케이션 특성상 홈에 기능이 모여있긴 했지만 예상보다 더 큰 결과였습니다.
  - 홈은 서버 컴포넌트인데, Next.js SSR 의 이점을 버리는게 훨씬 이득이었기에 홈에 있는 모든 컴포넌트를 클라이언트 컴포넌트로 분리하여 스플리팅 을 하였습니다. (서버 컴포넌트에서는 스플리팅이 되지 않았습니다.)
  - page.tsx의 용량이 7.2 MB 에서 2.1 MB 까지 줄었으며, LCP 를 4.2 에서 2.4까지 줄이는데 성공했습니다.

4. 개발경험

#3. 돈이 오가는 계좌 어플리케이션





감사합니다.